

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Report

Laboratory work No. 7

Three Small Programs: Processes, Redirection, a Pipe

Ciur Alexandrina, st. gr. FAF-241

Checked by:
Reițman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chișinău – 2026

Contents

1 LABORATORY OBJECTIVES AND THEORY **1**
1.1 Objective 1
1.2 Theoretical Background 1

2 PRACTICAL WORK AND OBSERVATIONS **2**
2.1 Part 1: Process Creation with fork, exec, and wait 2
2.1.1 Observe Answers (Part 1) 2
2.2 Part 2: Zombie Process Creation 3
2.2.1 Observe Answers (Part 2) 3
2.3 Part 3: Output Redirection 4
2.3.1 Observe Answers (Part 3) 4
2.4 Part 4: Pipeline Implementation 5
2.4.1 Observe Answers (Part 4) 5

3 CONCLUSIONS **6**

1 LABORATORY OBJECTIVES AND THEORY

1.1 Objective

The primary goal of this laboratory work is to explore POSIX process management mechanisms in Linux, including process creation via `fork()`, executable replacement using `execlp()`, process synchronization with `waitpid()`, identification of zombie process states, and standard file descriptor manipulation for output redirection and inter-process communication using pipes.

1.2 Theoretical Background

In UNIX-like operating systems, process creation and I/O redirection follow fundamental POSIX design principles:

- **Process Birth (`fork` + `exec`):** A new process is created by cloning the current process using `fork()`. The child process can subsequently invoke `exec()` to overwrite its memory image with a new binary executable.
- **Parent Synchronization & Zombie States:** When a child process terminates via `exit()`, its process table entry remains as a Zombie (`Z` state) until the parent retrieves its exit status using `wait()` or `waitpid()`.
- **File Descriptors & Redirection:** Every process inherits standard file descriptors: `0` (stdin), `1` (stdout), and `2` (stderr). System calls like `open()` allocate the lowest unused file descriptor index.
- **Inter-process Communication (Pipes):** A pipe created via `pipe()` provides a unidirectional data channel with a read end (`p[0]`) and a write end (`p[1]`).

2 PRACTICAL WORK AND OBSERVATIONS

2.1 Part 1: Process Creation with fork, exec, and wait

The program `run.c` demonstrates how a command shell executes commands by cloning itself, replacing the child image with `ls`, and waiting for execution completion.

```
$ gcc run.c -o run
$ ./run ls
child: pid=4821, parent=4820
pipeline  pipeline.c  redirect  redirect.c  run  run.c  zombie  zombie.c
parent: child 4821 exited with status 0
$ ./run date
child: pid=4824, parent=4823
Sat Oct 24 11:42:07 EEST 2026
parent: child 4824 exited with status 0
$ ./run nosuchprog
child: pid=4827, parent=4826
exec: No such file or directory
parent: child 4827 exited with status 127
```

Listing 2.1: Execution of run.c with successful and failing exec calls

2.1.1 Observe Answers (Part 1)

1. **Execution of `perror("exec")`:** The `execlp()` system call completely overwrites the child process image upon success, preventing subsequent instructions from executing. `perror("exec")` is reached only when `execlp()` fails (e.g., executing `nosuchprog`).

2. **Parent Process Identification:** The child prints the process ID of the parent process that invoked `fork()` (obtained via `getppid()`).

3. **Exit Status Comparison:** The parent observes an exit status of `0` for valid executables (`ls`, `date`) and `127` for non-existent programs.

2.2 Part 2: Zombie Process Creation

By delaying the parent's call to `waitpid()` using `sleep(20)`, the child process enters a zombie state upon completion.

```
$ ./zombie &
[1] 4902
parent 4902 sleeping, child 4903 has exited
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   3110    3102 Ss   bash
   4902    3110 S    ./zombie
   4903    4902 Z    [zombie] <defunct>
   4905    3110 R+   ps -o pid,ppid,stat,cmd
$ # after 20 seconds
parent: reaped child 4903
[1]+  Done                    ./zombie
$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
   3110    3102 Ss   bash
   4910    3110 R+   ps -o pid,ppid,stat,cmd
```

Listing 2.2: Process table inspection showing zombie process state

2.2.1 Observe Answers (Part 2)

1. **Process State Designation:** The letter `Z` (or `Z+`) in the `STAT` column indicates a Zombie (defunct) process that has completed execution but awaits parent collection.

2. **Zombie Cleanup Mechanism:** After 20 seconds, the zombie entry is removed from the process table when `waitpid(pid, &status, 0)` reads the exit status.

2.3 Part 3: Output Redirection

By closing file descriptor `1` before opening `out.txt`, standard output is redirected to disk.

```
$ gcc redirect.c -o redirect
$ ./redirect
$ cat out.txt
pipeline
pipeline.c
redirect
redirect.c
run
run.c
zombie
zombie.c
```

Listing 2.3: Output redirection execution and file inspection

2.3.1 Observe Answers (Part 3)

1. **File Descriptor Allocation:** `open()` assigned descriptor `1` because Linux always allocates the lowest available unassigned integer, and descriptor `1` was explicitly closed.

2. **Screen Output Suppression:** Output listing does not appear on the terminal because descriptor `1` points directly to `out.txt`.

3. **Standard Error Redirection:** To redirect error messages (`2> err.txt`), descriptor `2` (`stderr`) would be closed prior to calling `open()`.

2.4 Part 4: Pipeline Implementation

Inter-process communication between `ls /etc` and `wc -l` is achieved using a pipe.

```
$ gcc pipeline.c -o pipeline
$ ./pipeline
238
$ ls /etc | wc -l
238
```

Listing 2.4: Pipeline execution comparing custom C program and shell output

2.4.1 Observe Answers (Part 4)

1. **Pipeline Output Verification:** Both `./pipeline` and `ls /etc | wc -l` produced identical line count results.

2. **Hanging Child Process:** Commenting out `close(p[1])` in the parent causes the second child (`wc`) to hang indefinitely.

3. **Root Cause Analysis:** The `wc` utility blocks while waiting for an End-Of-File (EOF) on `p[0]`, which never arrives because the parent process retains an open write descriptor `p[1]`.

3 CONCLUSIONS

In this laboratory work, I gained practical experience with POSIX process creation, synchronization, and file descriptor manipulation in Linux. Implementing `run.c` demonstrated the core execution cycle of command shells through `fork()`, `exec()`, and `waitpid()`.

Furthermore, observing zombie processes in `zombie.c` highlighted kernel process table management requirements. Manipulating file descriptors in `redirect.c` and `pipeline.c` illustrated how UNIX systems implement output redirection and inter-process communication pipelines through low-level descriptor swapping.